QUBE - Queue-Based Unified Balancing Engine: Project Report

Project report · 1 November 2026 · prepared with Claude

1. Summary

QUBE is a C++17 simulator of a multi-counter waiting room with priority scheduling. Customers of three priority classes wait in three separate first-in-first-out lanes. Four service counters, each with a home class that can be changed while the program runs, pick their next customer with one rule that combines priority and waiting time (aging). The scheduling engine is a separate, tested module; a raylib front end shows the room as an animated isometric 3D scene with a live dashboard.

Key results from an engine-only simulation of one hour of the default workload:

- With the Aging policy, urgent (P3) customers waited 3.6 s on average and high-priority (P2) customers 5.2 s, against 6.9 s for normal (P1) customers. First-come-first-served gave every class about 6 s.
- The average wait over all customers stayed at 6.0 s, and nobody waited longer than 60 s (the starvation threshold).
- 23 automated engine tests pass. A simplified rewrite of the engine matched the original exactly on 400 random simulations.

2. Problem statement

Banks, hospitals, government offices and computer servers share one problem: many users arrive at random times, there are only a few service points, and not every user is equally urgent.

Two simple solutions both fall short:

- A single first-come-first-served queue treats an emergency like a routine request, so urgent users wait as long as everyone else.
- A strict priority rule serves urgent users first, but a steady stream of urgent users can leave normal users waiting indefinitely. This is called starvation.

Reordering one shared queue every time an urgent user arrives also makes the line hard to follow for the people in it and costly to maintain.

The problem addressed by this project is therefore to design and evaluate a scheduling method for several service counters that:

1. serves higher-priority customers sooner;
2. never starves a lower-priority class;
3. keeps the counters busy, without leaving a counter idle while someone waits;
4. keeps each customer in a stable position instead of reshuffling the queue; and
5. can be watched live and measured with clear statistics.

Success is judged by average, longest and per-class waiting time, the number of starved customers, fairness between classes and the utilization of the counters (section 9).

3. Objectives

The original brief asked for:

- a C++17 scheduling engine, kept separate from the rendering code;
- a realistic isometric 2.5D/3D waiting-room simulation with animated customers, counters and staff, not a console program;
- FCFS, priority, shortest-service-time and aging scheduling, with aging used to prevent starvation;
- the ability to add customers, set priorities and service durations, choose a policy, and start, pause, resume and reset;
- a dark dashboard with waiting and served counts, average and longest wait, counter status and utilization, throughput, completed customers, starvation and fairness statistics.

Later requirements were: three separate priority lanes instead of one queue, four counters with primary class assignments that may serve other classes when idle, starvation prevention through the existing aging mechanism, a 3D view that shows each lane and each counter's class, and an on-screen settings window.

4. Methodology

The work followed an incremental, test-driven approach: a small working prototype first, then one feature per stage, with each stage checked before the next began.

Phase	Activity	Result
Analyse	List the required features and the real-life situations the system should model	Requirements (section 3)
Design	Separate the scheduling engine from the interface; choose three lanes, home classes and aging	Architecture (section 6) and selection rule (section 7)
Prototype	Build the isometric scene with moving customers before adding any scheduling	A visible, working prototype
Implement	Add controls, the four policies, the fairness statistics, lanes and settings, one stage at a time	Six stages (section 11)
Test	Automated engine tests, a side-by-side comparison of two engine versions, and hour-long simulations	Results in section 10
Evaluate and tune	Compare the policies on the same random workload; sweep the aging override time	Default override of 6 s
Refine	Fix the problems found and simplify the code	Problems table in section 11

4.1 Simulation model

Time advances in fixed slices of at most 0.05 simulated seconds, so results do not depend on the frame rate or the speed setting. Customers arrive at random (the defaults are in section 7.6); each needs 4 to 12 s of service, preceded by a 3 s walk from the lane to the desk. A counter moves through the phases idle, walking and serving. The engine itself is deterministic: the same arrivals always give the same result, which makes comparisons repeatable.

4.2 Evaluation method

Each policy is run on the same random arrivals, so differences come from the policy and not from luck. The measures are the metrics in section 9. Engine-only runs, without graphics, simulate an hour in a fraction of a second, which made parameter sweeps practical; the graphical program shows the same engine live. The verification steps are in section 10.

5. Concepts used

5.1 Object-oriented programming in C++

Concept	Where it appears in QUBE
Class and object	`class Engine` holds the whole simulation; `main()` creates one `Engine` object.
Encapsulation	Public functions are what the interface may call; the data and helpers (`lanes_`, `dispatch`, `pickFor`) are private.
Constructors and overloading	`Engine()` and `Engine(Config)` are two constructors; each leaves the object ready to use by calling `reset()`.
Structs as data	`Customer`, `Counter`, `Config` and `Stats` are plain bundles of data with default values.
Enumerations	`enum class Policy`, `CustomerState` and `CounterPhase` name the allowed choices and prevent invalid values.
Methods and const	Functions marked `const` only read (`stats()`, `peekNext()`); `const &` returns avoid copying large lists.
Abstraction and modularity	The engine has no graphics code and the interface only calls its public functions, so each part can be changed or tested separately.

Inheritance and virtual functions are not used. A natural extension would be a base class `Policy` with one subclass per selection rule.

5.2 Other C++ features

- `std::vector` for customers, counters, lanes and waits, and `std::sort` for the percentile.

- A header file (`.hpp`) for the interface and a source file (`.cpp`) for the implementation, grouped in `namespace qube`.
- References, `const` and default member values.
- CMake for building, and a small test program with its own checks.

5.3 Scheduling and algorithm concepts

- FIFO queues, and priority scheduling with several queues.
- Shortest-job-first scheduling.
- Aging and starvation: a waiting score that grows with time (section 7.3).
- Non-preemptive scheduling: a customer at a desk is never interrupted.
- Work-conserving service: an idle counter helps another lane (borrowing).
- Load balancing: the least-busy idle counter acts first.
- Discrete-time simulation with fixed time slices.
- Fairness and tail latency: Jain's index and the 90th-percentile wait.

5.4 Graphics and interface concepts

- The game loop: read input, update the engine, draw.
- An orthographic (isometric) camera, and projection from 3D to the screen for the floating labels.
- Depth testing and simple lighting by face direction.
- Animation along waypoints, with walking speed matched to the engine's timer.
- An immediate-mode interface: buttons are drawn and tested every frame.
- Settings saved to a text file between runs.

6. System architecture

Layer	Files	Responsibility
Scheduling engine	`src/engine/Engine.hpp`, `Engine.cpp`	Customers, lanes, counters, the selection rule, time stepping and all statistics. No graphics code.
Renderer and interface	`src/main.cpp`	Isometric 3D scene, customer animation, dashboard, control bar, settings window. Reads the engine and calls its public functions.
Tests	`tests/engine_tests.cpp`	Automated checks of the engine.
Build	`CMakeLists.txt`	Downloads raylib 5.5 automatically and builds `qube.exe` and `qube_tests.exe`.

Each frame the program reads input, advances the engine with `step(dt)`, reads the engine's state and draws it. The engine never calls the renderer. Customer positions and walking animation exist only in the renderer; the engine knows only each customer's state. The engine times the walk from lane to desk (the approach time), and the renderer adjusts walking speed so that a customer arrives exactly when service begins.

raylib was chosen as the simplest practical route on Windows with VS Code. It is one library that CMake downloads and builds, it needs no asset files, and it supports an orthographic 3D camera, which gives a true isometric view. The characters, desks and floor are drawn from boxes and spheres in code, with custom per-face shading because raylib's built-in boxes are unlit.

7. Scheduling design

7.1 Lanes and counters

A waiting customer stands in the lane for their priority (P1, P2 or P3) and stays there until called. A lane is never re-sorted, so a new urgent customer does not move anyone else. Each counter has a home class: by default counter 1 is P3, counter 2 is P2 and counters 3 and 4 are P1. Clicking a counter's class badge cycles it through P1, P2 and P3.

7.2 The selection rule

When a counter is free, only the front customer of each lane is considered. Under the default Aging policy the rule has three stages, and the first one that finds a customer wins:

1. **Aging override.** Any lane front that has waited at least the override time (6 s by default) may be taken by any counter. If several qualify, the highest aging score goes first.
2. **Home lane.** Otherwise the counter takes the front customer of its own lane.
3. **Borrow.** If its home lane is empty, it takes the front customer with the best aging score from the other lanes.

No waiting customer is left unserved while a counter is idle, and a customer already at a desk is never interrupted.

7.3 The aging formula

For a customer with priority `p` who arrived at time `a`, the score at time `t` is `E = p + (t - a) / T`, where `T` is the aging interval (15 s by default). Waiting `T` seconds is worth one priority level. Rewriting it as `E = (t - V) / T` with `V = a - T*p` shows that the ranking depends only on a fixed 'virtual arrival time' `V`: each priority level counts as arriving `T` seconds earlier. This is why lanes never need re-sorting, and why comparing three lane fronts gives the same answer as comparing every waiting customer.

A customer who arrived `d` seconds before a customer `Δp` levels higher outranks them when `d > T * Δp`. With the defaults, a waiting P1 beats a fresh P2 after 15 s more waiting, and a fresh P3 after 30 s more. This bounds how long a lower class can be overtaken, which is how starvation is prevented.

7.4 Policies

The four policies are selectable in the top bar or with keys 1 to 4:

Policy	Rule	Purpose
FCFS	Oldest lane front overall; ignores lanes and home classes.	Baseline

Policy	Rule	Purpose
Priority	Home lane first; if empty, the highest class waiting. No aging.	Shows how priority alone can starve a class
SJF	Shortest service time among everyone waiting; ignores home classes.	Baseline for efficiency
Aging (default)	The three-stage rule in section 7.2.	The proposed design

7.5 Choosing the counter

When several counters are idle, those whose own lane has someone waiting act first (Priority and Aging only). Ties go to the counter that has worked the least, which spreads the load evenly. An earlier version always chose the lowest-numbered idle counter, which overloaded counter 1.

7.6 Default settings

Setting	Default
Counters	4 (P3, P2, P1, P1)
Arrivals	One customer about every 5.5 s (0.6 to 1.4 times), plus a rush of about 6 people every 45 s
Service time	4 to 12 s, in 0.5 s steps
Class mix	15% P3, 25% P2, 60% P1
Aging override wait	6 s
Aging interval	15 s per priority level
Starvation threshold	60 s
Walk to desk	3 s

All of these can be changed in the Settings window and are saved between runs.

8. Visualization and user interface

Figures 1 to 3 show the running program. They were captured in a busier session than the default workload used in section 10.3, so the waits in them are higher than in that table.

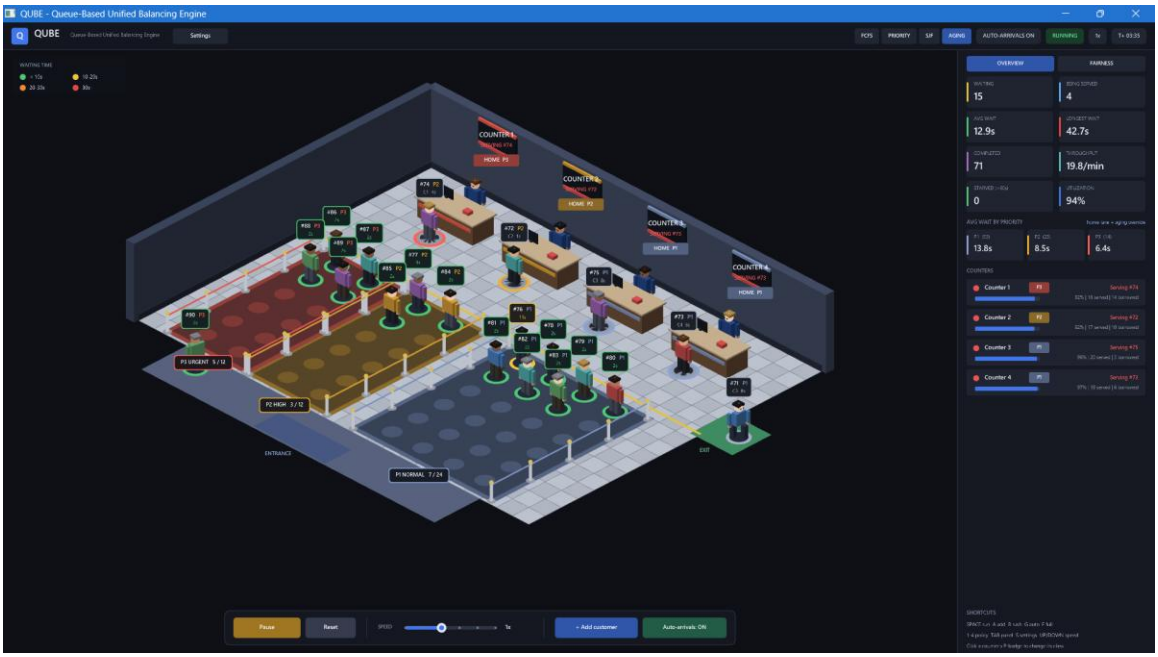


Figure 1. QUBE running the Aging policy 3 min 35 s into a run: three priority lanes (P3, P2, P1) feed four counters, with the Overview statistics on the right.

- **Scene.** A tiled waiting room with three coloured lanes (P3 red, P2 amber, P1 blue) with rope barriers and slot markers, an entrance corridor, an exit, and four desks along the back wall with animated staff. Customers are 3D figures that walk in, wait in their lane, walk to a counter, are served and leave.
- **Customer labels.** Each customer shows an ID, a coloured priority tag and a live timer. The ring under their feet is coloured by waiting time: green under 10 s, yellow under 20 s, orange under 30 s, red beyond.
- **Counters.** Each counter sign shows its status (open, calling, serving) and a clickable class badge.
- **Dashboard (Overview tab).** Waiting, being served, average and longest wait, completed, throughput per minute, starved customers, utilization, average wait per priority, and a card per counter with class, status, utilization, number served and number borrowed.
- **Dashboard (Fairness tab).** Jain fairness index, wait standard deviation, 90th-percentile wait, the P1/P3 wait ratio, counter balance, starved customers per class, wait-by-priority bars and a chart of queue length over the last five simulated minutes.
- **Controls.** A bar with Start, Pause, Resume, Reset, a speed slider (0.25x to 8x), an auto-arrivals toggle and an add-customer form (priority, service time, quantity).
- **Settings window.** Arrival interval, rush frequency and size, service-time range, class mix, aging override, aging interval, starvation threshold, number of counters (2 to 6) and walk time.
- **Keys.** Space start, pause or resume; A add a customer; B rush; G auto-arrivals; 1 to 4 policy; Tab switch dashboard tab; S settings; R reset; F fullscreen.

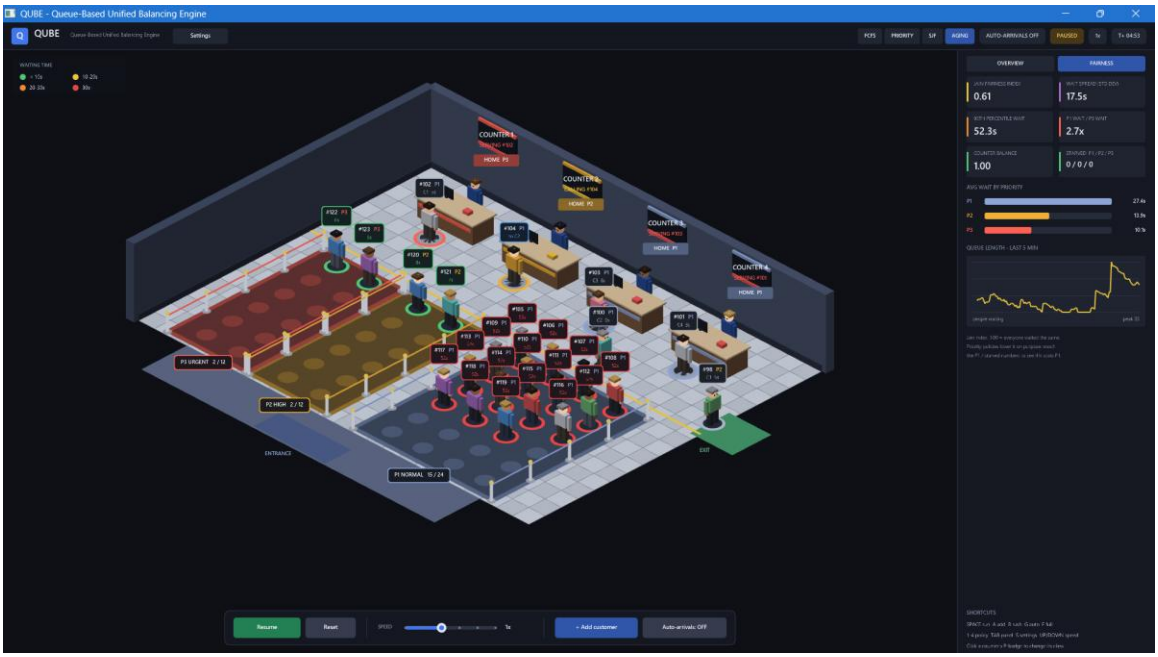


Figure 2. The Fairness tab with the simulation paused at 4 min 53 s: Jain index 0.61, P1 customers waiting 2.7 times longer than P3, and the queue length over the last five simulated minutes.

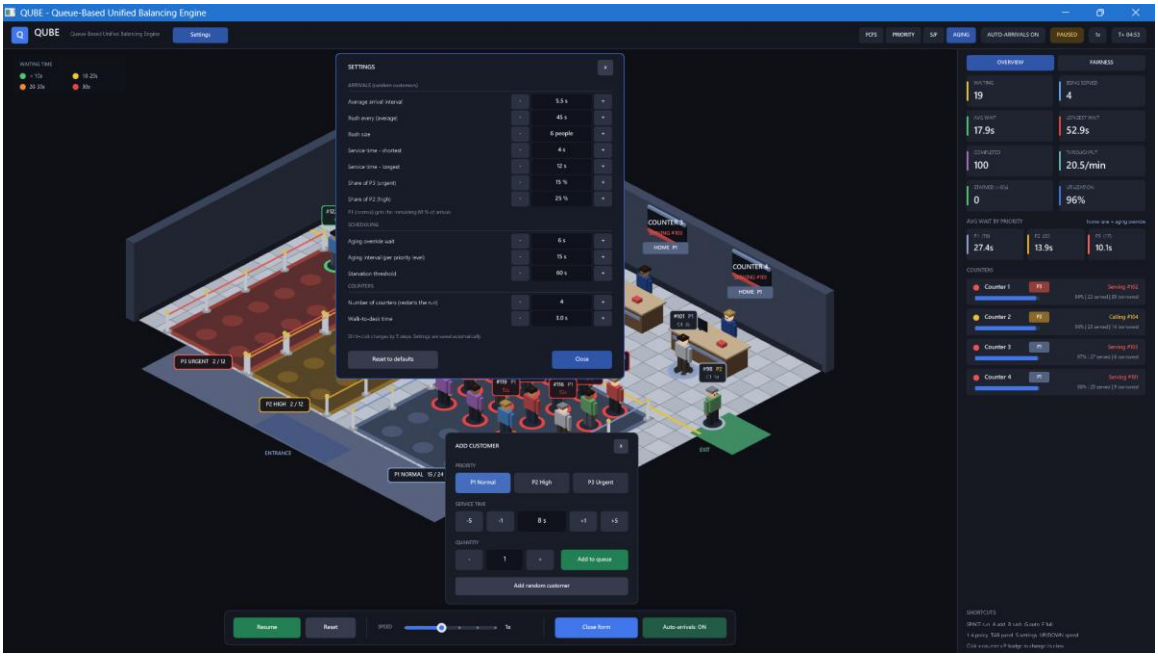


Figure 3. The Settings window and the add-customer form open together, with the simulation paused.

9. Metrics

Metric	Definition
Wait	Time from arrival until a counter calls the customer; for someone

Metric	Definition
	still waiting, the wait so far.
Starvation	A customer whose wait exceeds the threshold (counted once, per class as well).
Utilization	A counter's walking plus serving time divided by elapsed time.
Throughput	Completed customers per minute.
Jain fairness index	`(sum of waits)^2 / (n * sum of squared waits)`; 1.0 means everyone waited equally.
Counter balance	The same index applied to the counters' busy times; 1.0 means perfectly even load.
Priority gap	Average P1 wait divided by average P3 wait.
Wait spread and p90	Standard deviation of waits and the 90th-percentile wait.

Priority policies lower the Jain index on purpose, because they treat classes unequally; the fairness tab is meant to show what that costs the lowest class.

10. Verification and results

10.1 Automated tests

`tests/engine_tests.cpp` contains 23 test functions, all passing. They cover FCFS, priority and SJF ordering; parallel counters and timing; independence from step size (one 40 s step against 4,000 steps of 0.01 s); lanes staying stable when a higher class arrives; home counters taking their own lane; idle counters borrowing; the aging override; starvation under Priority and its prevention under Aging when every counter is a P3 counter; counter reassignment surviving a reset; load balancing; the fairness statistics; and live changes to settings and to the number of counters.

The graphical program has no automated visual tests. It was checked by building and running it on a Windows laptop at each stage.

10.2 Equivalence of the simplified engine

The engine was later rewritten in simpler, beginner-level C++ (plain loops, one function per selection rule, no lambdas or pointers). The old and new engines were run side by side on 400 random simulations with all four policies, 1 to 6 counters and changes to policy, counter classes and settings during the run. Every customer, counter, lane and statistic was compared: 25,807 customers, no differences.

10.3 Simulation results

One simulated hour of the default workload on four counters, engine only, one random seed. Waits are in seconds.

Policy	P1	P2	P3	Average	Longest	Starved	Customers
FCFS	5.9	6.4	5.5	6.0	25.1	0	1,156
Priority	6.4	6.1	4.2	6.0	36.1	0	1,156
SJF	5.1	5.7	4.5	5.2	38.0	0	1,155
Aging	6.9	5.2	3.6	6.0	28.1	0	1,154

Counter utilization was about 87% under every policy. Compared with FCFS, Aging cut the P3 wait by about 35% and the P2 wait by about 19%, at the cost of about 17% more for P1, with the overall average unchanged. Priority without aging helped only P3 and let the longest wait rise to 36.1 s. SJF had the lowest average but the longest waits and no regard for priority.

10.4 Tuning the aging override

With a 30 s override, P2 waited almost as long as P1, because the P1 counters stayed with their own lane. Over two random seeds, a 6 s override gave P1 6.9 to 7.7 s, P2 4.6 to 5.2 s and P3 3.6 to 4.2 s, a clear ordering, so 6 s became the default. It was tuned on this workload only and is adjustable in the Settings window.

10.5 Performance

The simplified engine advances time as fast as the original. Its `stats()` function is slower because it now sorts waits for the 90th percentile: about 500 to 580 microseconds per call with 11,322 customers (roughly ten simulated hours), against about 150 microseconds before. That is about 3% of a 60 frames-per-second frame, and the cost grows with the number of customers.

11. Development history and problems solved

Stage	Work
1	Engine with FCFS and the first isometric prototype with a basic dashboard.
2	On-screen controls, speed slider and add-customer form.
3	Priority, SJF and Aging policies, policy selector, per-priority waits, starvation count.
4	Fairness tab and load balancing across counters.
5	Three priority lanes, counters with home classes, borrowing and the aging override.
6	Settings window with saved values, and a beginner-level rewrite of the engine.

Problems found and fixed along the way:

Problem	How it was found	Fix
Arrivals exceeded capacity, so the queue grew without limit (105 waiting)	A screenshot from the first run	Lower arrival rate and a cap on auto-arrivals

Problem	How it was found	Fix
After that, almost no queue formed (average wait 0.1 s), so priority had nothing to reorder	Headless simulation	Periodic rushes of customers
Counter 1 did most of the work at low load	Reviewing the counter-selection code	Choose the idle counter that has worked least
Each phase ran one extra 0.05 s slice because of floating-point round-off	A percentile test	A 1e-9 tolerance on the timers
P2 barely beat P1 with a 30 s override	Parameter sweep	Default override lowered to 6 s
Several test scenarios were wrong (for example an idle counter correctly borrowed the customer the test expected to starve)	Failing tests	Test setups corrected, engine unchanged

12. Limitations

- A customer's lane is fixed at arrival; aging changes who is picked, not which lane someone stands in.
- Service is non-preemptive: an urgent customer may wait for a counter that is serving someone else.
- SJF considers everyone waiting, so a customer can leave from the middle of a lane.
- Results come from random simulated arrivals, one seed per policy comparison, with no confidence intervals and no real-world data.
- The 6 s override default was tuned on one workload.
- The program was built and run on Windows only, and customer labels overlap when a lane is crowded.
- The camera is fixed.

13. Future work

- Hover-only customer labels and camera zoom and pan.
- Several random seeds with confidence intervals, and export of results to a file.
- Optional preemption or promotion of long-waiting customers between lanes.
- A downloadable Windows build published on GitHub.

14. Build and run

On Windows with Git, CMake and MinGW-w64 (WinLibs) installed:

```
cmake -S . -B build -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release "-
DCMAKE_POLICY_VERSION_MINIMUM=3.5"
cmake --build build -j
.\build\qube_tests.exe
.\build\qube.exe
```

The first configure downloads raylib and needs an internet connection. The program opens in a Ready state; press Start or Space to begin.

Appendix: source files

File	Role
`src/engine/Engine.hpp`	Data types and the Engine interface (161 lines)
`src/engine/Engine.cpp`	Selection rules, time stepping, statistics (448 lines)
`src/main.cpp`	Renderer, characters, dashboard, controls, settings window (over 1,000 lines)
`tests/engine_tests.cpp`	23 engine tests
`CMakeLists.txt`	Build configuration; downloads raylib 5.5
`README.md`	Setup, controls and layout

15. Engine Code:

```
#include "engine/Engine.hpp"

#include <algorithm>  // std::sort, std::max, std::min

#include <cmath>      // std::sqrt

namespace qube {

// A small helper (not part of the Engine): keeps a number between low and high.

static int limit(int value, int low, int high) {

    if (value < low) return low;

    if (value > high) return high;

    return value;

}

// ================================================================== creating and
resetting

// In this file every function name starts with "Engine::" - it tells the compiler

// "this function belongs to the Engine class".

Engine::Engine() {

    reset();

}
```

```
Engine::Engine(Config config) {
    config_ = config;
    reset();
}
void Engine::reset() {
    now_ = 0.0;
    customers_.clear();
    for (int i = 0; i < 3; i++) {
        lanes_[i].clear();
    }
    int count = config_.counterCount;
    if (count < 1) count = 1;
    config_.homeClasses.resize(count, 1);   // one home class per counter (new ones default to
P1)
    counters_.clear();
    for (int i = 0; i < count; i++) {
        Counter counter;
        counter.id = i;
        counter.homeClass = limit(config_.homeClasses[i], 1, 3);
        counters_.push_back(counter);
    }
}
// ============================================================================ customers
int Engine::addCustomer(int priority, double serviceTime) {
    Customer c;
    c.id = (int)customers_.size() + 1;
    c.priority = limit(priority, 1, 3);
    c.serviceTime = std::max(0.5, serviceTime);
    c.arrival = now_;
    customers_.push_back(c);

    // The customer joins the back of the lane for their priority and stays there until
called.
    lanes_[c.priority - 1].push_back(c.id);
```

```
    return c.id;
}
const Customer& Engine::customer(int id) const {
    return customers_.at(id - 1);          // ids start at 1, the list starts at 0
}
const std::vector<int>& Engine::lane(int cls) const {
    return lanes_[limit(cls, 1, 3) - 1];
}
// ============================================================================ settings
void Engine::setAgingOverrideWait(double seconds) {
    if (seconds < 0.0) seconds = 0.0;
    config_.agingOverrideWait = seconds;
}
void Engine::setAgingInterval(double seconds) {
    if (seconds < 1.0) seconds = 1.0;
    config_.agingInterval = seconds;
}
void Engine::setStarvationThreshold(double seconds) {
    if (seconds < 1.0) seconds = 1.0;
    config_.starvationThreshold = seconds;
}
void Engine::setApproachTime(double seconds) {
    if (seconds < 0.5) seconds = 0.5;
    config_.approachTime = seconds;
}
void Engine::setCounterClass(int counterIndex, int cls) {
    Counter& counter = counters_.at(counterIndex);
    counter.homeClass = limit(cls, 1, 3);
    config_.homeClasses[counterIndex] = counter.homeClass;   // remembered even after reset()
}

void Engine::setCounterCount(int count) {
    config_.counterCount = limit(count, 1, 8);
```

```
    config_.homeClasses.resize(config_.counterCount, 1);
    reset();
}
// ============================================================================ small helpers
int Engine::frontOfLane(int cls) const {
    const std::vector<int>& l = lanes_[cls - 1];
    if (l.empty()) return 0;
    return l[0];
}
double Engine::waitOf(int id) const {
    return now_ - customer(id).arrival;
}
// Aging score = priority + waited / agingInterval.
// Everyone gains score at the same speed while waiting, so a P1 who has waited long enough
// can eventually beat a P3 who has just arrived.
double Engine::agingScore(int id) const {
    double interval = config_.agingInterval;
    if (interval <= 0.0) interval = 1.0;
    return customer(id).priority + waitOf(id) / interval;
}
bool Engine::anyWaiting() const {
    for (int i = 0; i < 3; i++) {
        if (!lanes_[i].empty()) return true;
    }
    return false;
}
void Engine::removeFromLane(int id) {
    std::vector<int>& l = lanes_[customer(id).priority - 1];
    for (int i = 0; i < (int)l.size(); i++) {
        if (l[i] == id) {
            l.erase(l.begin() + i);
            return;
        }
```

```
    }
}
// ========================================================================== the selection rules
// Every rule answers one question: "WHICH waiting customer should this counter take?"
// They only read data (const) and return a customer id, or 0 if nobody is waiting.

int Engine::peekNext(int counterIndex) const {
    return pickFor(counters_.at(counterIndex));
}
int Engine::pickFor(const Counter& counter) const {
    if (config_.policy == Policy::FCFS) return pickOldestFront();
    if (config_.policy == Policy::SJF) return pickShortestJob();
    if (config_.policy == Policy::Priority) return pickByPriority(counter);
    return pickByAging(counter);           // Policy::Aging
}
// FCFS: whoever arrived first, looking at the three lane fronts.
int Engine::pickOldestFront() const {
    int best = 0;
    for (int cls = 1; cls <= 3; cls++) {
        int front = frontOfLane(cls);
        if (front == 0) continue;
        if (best == 0) {
            best = front;
        } else if (customer(front).arrival < customer(best).arrival) {
            best = front;
        } else if (customer(front).arrival == customer(best).arrival && front < best) {
            best = front;                  // same arrival time: the smaller id wins
        }
    }
    return best;
}
// SJF: the shortest service time among EVERYONE waiting (not only the fronts).
int Engine::pickShortestJob() const {
```

```
    int best = 0;
    for (int cls = 1; cls <= 3; cls++) {
        const std::vector<int>& l = lanes_[cls - 1];
        for (int i = 0; i < (int)l.size(); i++) {
            int id = l[i];
            if (best == 0) {
                best = id;
            } else if (customer(id).serviceTime < customer(best).serviceTime) {
                best = id;
            } else if (customer(id).serviceTime == customer(best).serviceTime && id < best) {
                best = id;
            }
        }
    }
    return best;
}
// Priority: own lane first; if it is empty, the highest class that has someone waiting.
int Engine::pickByPriority(const Counter& counter) const {
    int homeFront = frontOfLane(limit(counter.homeClass, 1, 3));
    if (homeFront != 0) return homeFront;
    for (int cls = 3; cls >= 1; cls--) {
        int front = frontOfLane(cls);
        if (front != 0) return front;
    }
    return 0;
}

// Aging (the default policy) has three stages. The first one that finds someone wins.
int Engine::pickByAging(const Counter& counter) const {
    int best = 0;
    double bestScore = -1e18;
    // Stage 1 - OVERRIDE: a lane front that has already waited long enough may be taken
    // by ANY counter. If several qualify, the one with the highest aging score goes first.
```

```
    for (int cls = 3; cls >= 1; cls--) {
        int front = frontOfLane(cls);
        if (front == 0) continue;
        if (waitOf(front) < config_.agingOverrideWait) continue;   // has not waited long
enough
        double score = agingScore(front);
        if (score > bestScore) {
            bestScore = score;
            best = front;
        }
    }
    if (best != 0) return best;
    // Stage 2 - HOME LANE: take the front customer of this counter's own lane.
    int homeFront = frontOfLane(limit(counter.homeClass, 1, 3));
    if (homeFront != 0) return homeFront;

    // Stage 3 - BORROW: the home lane is empty, so help another lane: best aging score wins.
    for (int cls = 3; cls >= 1; cls--) {
        int front = frontOfLane(cls);
        if (front == 0) continue;
        double score = agingScore(front);
        if (score > bestScore) {
            bestScore = score;
            best = front;
        }
    }
    return best;
}
// ========================================================================== moving time forward
void Engine::step(double seconds) {
    // Time is cut into slices of at most 0.05 s, so the result does not depend on frame rate.
    const double maxSlice = 0.05;
    while (seconds > 1e-9) {
        double slice = std::min(seconds, maxSlice);
```

```
        advance(slice);
        seconds -= slice;
    }
}
void Engine::advance(double seconds) {
    now_ += seconds;
    for (int i = 0; i < (int)counters_.size(); i++) {
        updateCounter(counters_[i], seconds);
    }
    dispatch();                             // idle counters may now take new customers
}
// Counts down one counter's timer and moves it to its next phase when the timer reaches zero.
void Engine::updateCounter(Counter& counter, double seconds) {
    if (counter.phase == CounterPhase::Idle) return;

    counter.occupiedTime += seconds;
    counter.timer -= seconds;
    if (counter.timer > 1e-9) return;      // phase not finished yet (tiny tolerance for
rounding)
    Customer& c = customers_[counter.customerId - 1];
    if (counter.phase == CounterPhase::Approaching) {
        // The customer reached the desk: service begins.
        counter.phase = CounterPhase::Serving;
        counter.timer = c.serviceTime;
        c.state = CustomerState::Serving;
        c.startedAt = now_;
    } else {
        // Service is over: the customer is done and the counter is free again.
        c.state = CustomerState::Done;
        c.finishedAt = now_;
        counter.phase = CounterPhase::Idle;
        counter.customerId = 0;
        counter.timer = 0.0;
        counter.served++;
```

```
        if (c.priority != counter.homeClass) counter.servedForeign++;   // it "borrowed" this
customer
    }
}
int Engine::chooseIdleCounter() const {
    bool homeAware = (config_.policy == Policy::Priority || config_.policy == Policy::Aging);
    int best = -1;
    bool bestHasHomeWork = false;

    for (int i = 0; i < (int)counters_.size(); i++) {
        const Counter& counter = counters_[i];
        if (counter.phase != CounterPhase::Idle) continue;

        bool hasHomeWork = homeAware && !lanes_[counter.homeClass - 1].empty();
        bool better = false;
        if (best == -1) {
            better = true;
        } else if (hasHomeWork && !bestHasHomeWork) {
            better = true;
        } else if (hasHomeWork == bestHasHomeWork &&
                   counter.occupiedTime < counters_[best].occupiedTime - 1e-9) {
            better = true;
        }
        if (better) {
            best = i;
            bestHasHomeWork = hasHomeWork;
        }
    }
    return best;
}
void Engine::dispatch() {
    while (anyWaiting()) {
        int counterIndex = chooseIdleCounter();
        if (counterIndex == -1) break;                 // every counter is busy
```

```
        Counter& counter = counters_[counterIndex];
        int id = pickFor(counter);
        if (id == 0) break;

        removeFromLane(id);
        Customer& c = customers_[id - 1];
        c.state = CustomerState::Approaching;
        c.calledAt = now_;
        c.counter = counter.id;

        counter.phase = CounterPhase::Approaching;
        counter.customerId = id;
        counter.timer = config_.approachTime;          // time to walk from the lane to the
desk
    }
}
Stats Engine::stats() const {
    Stats s;
    s.total = (int)customers_.size();
    s.elapsed = now_;
    double waitSum = 0.0;                  // waits of customers who were already called
    int waitCount = 0;
    double priorityWaitSum[3] = {0.0, 0.0, 0.0};
    std::vector<double> waits;             // one wait per customer (for the fairness numbers)

    for (int i = 0; i < (int)customers_.size(); i++) {
        const Customer& c = customers_[i];

        // A customer's wait = time from arrival until called. If still waiting: the wait so
far.
        double wait = 0.0;
        if (c.state == CustomerState::Waiting) {
            s.waiting++;
```

```
            wait = now_ - c.arrival;
        } else if (c.state == CustomerState::Done) {
            s.completed++;
        } else {
            s.inService++;
        }
        if (c.calledAt >= 0.0) {
            wait = c.calledAt - c.arrival;
            waitSum += wait;
            waitCount++;
        }
        if (wait > s.longestWait) s.longestWait = wait;
        waits.push_back(wait);

        int p = limit(c.priority, 1, 3) - 1;               // 0, 1 or 2
        if (wait > config_.starvationThreshold) {
            s.starved++;
            s.starvedByPriority[p]++;
        }
        priorityWaitSum[p] += wait;
        s.countByPriority[p]++;
    }

    if (waitCount > 0) s.avgWait = waitSum / waitCount;
    for (int p = 0; p < 3; p++) {
        if (s.countByPriority[p] > 0) {
            s.avgWaitByPriority[p] = priorityWaitSum[p] / s.countByPriority[p];
        }
    }
    if (now_ > 1.0) s.throughputPerMin = s.completed / (now_ / 60.0);

    double utilizationSum = 0.0;
    for (int i = 0; i < (int)counters_.size(); i++) {
```

```
        utilizationSum += utilization(i);
    }
    if (!counters_.empty()) s.avgUtilization = utilizationSum / counters_.size();

    fillFairness(s, waits);
    return s;
}
// Jain's fairness index:  J = (sum of x)^2 / (n * sum of x^2).   1.0 means "all values are
equal".
void Engine::fillFairness(Stats& s, std::vector<double> waits) const {
    if (!waits.empty()) {
        double sum = 0.0;
        double sumOfSquares = 0.0;
        for (int i = 0; i < (int)waits.size(); i++) {
            sum += waits[i];
            sumOfSquares += waits[i] * waits[i];
        }
        double n = (double)waits.size();
        if (sumOfSquares > 1e-9) s.jainIndex = (sum * sum) / (n * sumOfSquares);

        double variance = sumOfSquares / n - (sum / n) * (sum / n);
        if (variance < 0.0) variance = 0.0;
        s.waitStdDev = std::sqrt(variance);

        std::sort(waits.begin(), waits.end());              // smallest first
        int index = (int)(0.9 * (waits.size() - 1));
        s.waitP90 = waits[index];
    }
    // How much longer do normal (P1) customers wait than urgent (P3) ones?
    if (s.countByPriority[0] > 0 && s.countByPriority[2] > 0 && s.avgWaitByPriority[2] > 0.05)
{
        s.priorityGap = s.avgWaitByPriority[0] / s.avgWaitByPriority[2];
    }
```

```
    // Is the work spread evenly over the counters? (same formula, applied to busy time)
    double sum = 0.0;
    double sumOfSquares = 0.0;
    for (int i = 0; i < (int)counters_.size(); i++) {
        sum += counters_[i].occupiedTime;
        sumOfSquares += counters_[i].occupiedTime * counters_[i].occupiedTime;
    }
    if (sumOfSquares > 1e-9) {
        s.counterBalance = (sum * sum) / (counters_.size() * sumOfSquares);
    }
}
double Engine::utilization(int counterIndex) const {
    if (now_ < 1e-6) return 0.0;
    const Counter& counter = counters_.at(counterIndex);
    return std::min(1.0, counter.occupiedTime / now_);
}

} // namespace qube
```